

Lösungsheft

Tag 9

Musterlösungen – Scrum, Kanban
und CI/IaC als Steuerungssystem.

Musterlösungen · nicht die einzige Antwort

Hinweis:

Musterlösungen sind Diskussionsbasis,
nicht die einzige korrekte Lösung.

Begleitend:

Aufgabenheft & Lehrskript Tag 9

Dozent:

Can Siebert

Niveau-Mix:

3 L1 · 5 L2 · 3 L3

Hinweise zu den Musterlösungen

Heute steht der Senior-Reflex im Vordergrund: *Agile ist Steuerung, nicht Meeting-Kalender*. Eine Lösung ohne klare Quality Gates, ohne Stop-the-Line-Regel und ohne Goodhart-Schutz für DORA-Kennzahlen ist nicht "Senior".

Nutzung im Coaching

Prüfen Sie Ihre Lösung gegen drei Fragen: (1) Habe ich Scrum/Kanban-Wahl mit 6 *Stichpunkten* begründet? (2) Hat meine Pipeline echte Stop-Regeln? (3) Habe ich DORA-Anti-Gaming eingebaut?

Niveau L1 – Wissen & Reproduktion

Hilfsvideos zu diesem Tag

Die folgenden YouTube-Erklärvideos vermitteln die Inhalte, die Sie zur Bearbeitung der Aufgaben benötigen. Klicken Sie auf den Titel, um das Video direkt zu öffnen; die vollständige URL steht in Klammern.

1. Scrum-Framework — Rollen, Events, Artefakte

<https://youtu.be/Ibz9STVjtzI>

(URL: <https://youtu.be/Ibz9STVjtzI>)

2. CI/CD-Pipelines — sichere und schnelle Änderungen

<https://youtu.be/xzV0KiprcZc>

(URL: <https://youtu.be/xzV0KiprcZc>)

3. Infrastructure as Code — Reproduzierbarkeit und Auditierbarkeit

https://youtu.be/3gZ_-k9t-KQ

(URL: https://youtu.be/3gZ_-k9t-KQ)

4. DORA-Kennzahlen — Lead Time, Deployment Frequency, MTTR, Change Failure Rate

<https://youtu.be/lqsENCje41w>

(URL: <https://youtu.be/lqsENCje41w>)

Tipp: Öffnen Sie das Video, lassen Sie es im Hintergrund laufen und arbeiten Sie parallel an der Aufgabe.

Aufgabe 1 – Scrum-Framework

L1 • Wissen

~ 8 min

YouTube-Suchquery: Scrum Framework Roles Events Artifacts Cargo Cult Scrum

Musterlösung

Drei Rollen:

- *Product Owner* – verantwortet *Was?* (Wert, Priorisierung, Akzeptanz).
- *Scrum Master* – verantwortet *Wie?* (Prozess, Hindernisse beseitigen, kein Team-Lead).
- *Developers* – verantworten *Lieferung* (Increment in Definition of Done).

Vier Events:

- *Sprint Planning* – *Was* im Sprint, *wie* lösen wir es.
- *Daily* – 15 Min Sync auf das Sprint Goal; keine Status-Show.
- *Review* – Increment den Stakeholdern zeigen, Feedback.
- *Retro* – Prozess verbessern, kein Schuldfinden.

Drei Artefakte:

- *Product Backlog* – priorisierte Liste aller Anforderungen.
- *Sprint Backlog* – Auswahl für den Sprint + Plan.
- *Increment* – fertig (Definition of Done) und potenziell auslieferbar.

Cargo Cult Scrum: Rituale werden gespielt (Dailies, Plannings), aber ohne Wirkung. Symptome: lange Meetings, kein klares Sprint Goal, Backlog wandert nicht; Review wird zur "Status-Show". Verlorener Wert: *empirische Steuerung* (Inspect & Adapt) – der Kern von Scrum ist weg.

Aufgabe 2 – Kanban-Prinzipien

L1 • Wissen

~ 8 min

YouTube-Suchquery: Kanban Principles Visualize WIP Limit Pull Flow Lead Time Cycle Time

Musterlösung

Prinzip	Wirkung	Fehlt es...
Visualisieren	Arbeit sichtbar, Bottlenecks erkennbar.	“unsichtbare Arbeit”, alles in Köpfen.
WIP limitieren	weniger Multitasking, kürzere Lead Time.	“alles parallel”, nichts wird fertig.
Pull-System	Team zieht, wenn Kapazität frei.	Push erzeugt “In Progress”-Stau.
Flow messen	Lead/Cycle Time als objektive Steuerung.	subjektive Bewertung, kein Lerneffekt.

WIP-Limit als Schutz, kein Aufwand-Hindernis:

Ein WIP-Limit zwingt das Team, *Ende* (Done) wichtiger zu nehmen als *Anfang* (Start). Multitasking erhöht Lead Time nachweislich – weniger ist mehr. Das WIP-Limit ist ein *Schutzmechanismus* für Qualität und Fokus, kein bürokratisches Hindernis.

Aufgabe 3 – CI-Pipeline & IaC

L1 • Wissen

~ 10 min

YouTube-Suchquery: CI Pipeline Build Test Deploy Quality Gate IaC Infrastructure as Code

Musterlösung

CI-Pipeline Stufen + Quality Gates:

Stufe	Was passiert?	Quality Gate
Commit	Code wird gepusht.	Pre-commit-Check (Linter, Format).
Build	Quellcode → Artefakt (z. B. Container Image).	Build erfolgreich, keine Warnings.
Tests	Unit + Integration.	Test-Quote $\geq$ Schwelle (z. B. 80 %).
Security/Quality	Static Analysis, Dependency Scan.	Keine kritischen Vulnerabilities.
Deploy Staging	Auf Staging-Umgebung.	Smoke Tests grün.
Deploy Prod	Auf Production.	Smoke Tests + Health-Check OK; Rollback-Plan dokumentiert.

Infrastructure as Code:

Was: Infrastruktur wird in Code-Dateien beschrieben (Terraform, Pulumi, Ansible) und versioniert. **Vorteile:** (a) reproduzierbar (gleiche Umgebung jedes Mal), (b) reviewbar (Pull Requests), (c) auditierbar (Versionshistorie). **Risiken:** (a) falsche Rechte (zu weit gefasst), (b) unkontrollierte Änderungen ohne Review (“Quick-Fix über Console” hängt nicht im Code).

Niveau L2 – Anwendung & Transfer

Aufgabe 4 – Scrum oder Kanban?

L2 • Anwendung

~ 25 min

YouTube-Suchquery: Scrum vs Kanban Hybrid Decision Process Improvement Operating Model

Musterlösung**1. Entscheidung: Hybrid. 6 Stichpunkte:**

- Risiko: Hybrid spreizt das Risiko – Sprints für Features, Kanban-Lane für Incidents.
- Planbarkeit: Sprints geben Stakeholdern Taktung.
- Stakeholder: viele Stakeholder → Review-Format nötig (Scrum-Review).
- Teamreife: Hybrid passt zu mittlerer Reife; reines Scrum würde Incidents stören.
- Workload: schwankender Anteil Anforderungsänderungen → Kanban-Lane absorbiert.
- Abhängigkeiten: gegenüber externen Stakeholdern: Sprint Review als “Verlässlichkeits-Anker”.

2. Ablauf-Skizze (2 Wochen, Hybrid):

- *Scrum-Lane (Features)*: Sprint Goal “MVP Genehmigungsweg 1 live”; 5 Backlog Items; DoD inkl. Tests + Doku + Audit-Log; Review-Agenda 60 Min.
- *Kanban-Lane (Incidents/Requests)*: Spalten Backlog → In Progress (WIP 2) → Review → Done; Pull-Regel “erst leeren, dann ziehen”; Replenishment-Meeting Mo + Mi.

3. Drei Policies (Verantwortungsfähigkeit):

1. *Klare Abnahme*: jedes Item hat einen genannten Approver vor “Done”.
2. *Eskalation*: blockierte Items nach 24 h → Scrum Master / Team-Lead.
3. *Transparenz*: Board und Backlog sind öffentlich (kein “Schatten-Board”).

Aufgabe 5 – CI-Pipeline & Quality Gates

L2 • Anwendung

~ 22 min

YouTube-Suchquery: CI CD Pipeline Quality Gates IaC Policy Rollback Canary Feature Toggle

Musterlösung**1. Minimal-Pipeline (6 Stufen):**

Build □ Unit Tests □ Integration Tests □ Security Scan □ Deploy Staging □ Smoke Tests.

2. Zwei Quality Gates:

1. *Gate 1 – Tests + Security:* alle Tests grün *und* kein kritischer Security-Finding offen.
2. *Gate 2 – Smoke Tests + Health:* Staging-Smoke-Test grün *und* Health-Check OK vor Production-Deploy.

3. IaC-Policy:

- *Änderungen via Pull Request:* keine Konsolen-Änderungen.
- *Vier-Augen-Prinzip:* mind. 2 Reviewer bei Production-Umgebung, Secrets, Berechtigungen.
- *Audit Trail:* Merge-Historie + Deploy-Log; 7 Jahre Aufbewahrung.
- *Secrets:* im Secrets-Manager, nie im Repo.

4. Bewusstes Risiko + Schutz:

- *MVP-Risiko:* Integration Tests nur für Happy Path – Edge Cases bleiben offen.
- *Schutz:* **Feature Toggle** für den neuen Genehmigungsweg + **Canary Deploy** (5 % Traffic erst); **Rollback**-Plan getestet (1 Klick = vorherige Version). So bleibt der “Blast Radius” begrenzt.

Aufgabe 6 – DORA-Kennzahlen

L2 • Anwendung

~ 20 min

YouTube-Suchquery: DORA Metrics Lead Time Deployment Frequency MTTR Change Failure Rate

Musterlösung

DORA-Kennzahlen:

KPI	Misst + Verhalten	Manipulation + Gegenkopplung
Lead Time for Changes	Commit → Production. Fördert kleine, häufige Änderungen.	Tasks splitten → Lead Time scheinbar kürzer. Gegenkopplung: Change Failure Rate.
Deployment Frequency	Wie oft pro Zeitraum. Fördert Routine im Deployen.	“Leer-Deploys” → Künstlich häufig. Gegenkopplung: Anzahl Production-Bugs.
MTTR	Wie schnell Wiederherstellung. Fördert Incident-Reife.	Incidents zu früh “geschlossen”. Gegenkopplung: Recurrence-Rate.
Change Failure Rate	% Deployments mit Incidents. Fördert Qualität.	Klein-Releases als “erfolgreich” zählen. Gegenkopplung: Inkl. Edge-Case-Failures, kein Smart-Filter.

Executive-KPIs: *Lead Time* und *Change Failure Rate*. Begründung: Sie messen *Geschwindigkeit überlebt nur, wenn Qualität stimmt*. *Intern:* Deployment Frequency (operative Routine) und MTTR (Incident-Reife) – wichtig, aber als Executive-Signal anfälliger für Manipulation.

Faustregel (DORA): “Elite” = Lead Time < 1 Tag, Deployment Frequency > 1/Tag, MTTR < 1 h, Change Failure Rate < 15 %.

Aufgabe 7 – IaC-Workflow

L2 • Anwendung

~ 20 min

YouTube-Suchquery: Infrastructure as Code GitOps Pull Request Review Secrets Management Audit

Musterlösung**1. IaC-Workflow:**

Branch (Entwickler erstellt) → *Änderung* (Terraform/Pulumi, Commit) → *Pull Request* (Beschreibung, Test-Plan) → *Review* (Reviewer prüft Plan-Output, Berechtigungen, Secrets) → *Merge* (nach Approval) → *Deploy* (automatisch via Pipeline, mit Lock zur Verhinderung paralleler Läufe).

2. Vier-Augen-Pflicht:

- Production-Umgebung.
- Secrets / Credential-Änderungen.
- IAM / Berechtigungen / Policies.
- Netzwerk (Security Groups, Firewall, VPN).

3. Secrets-Policy:

- Secrets liegen im Secrets-Manager (z. B. Vault).
- *Anti-Muster*: (a) “Secret im Repo” (Git-Historie für immer kompromittiert), (b) “Secret im Wiki” (kein Audit), (c) “Secret in Slack” (keine Rotation, kein Audit).
- Rotation: regelmäßig (z. B. alle 90 Tage); nach Kompromittierung sofort.

4. Audit Trail:

Auditor sieht: (a) Git-Historie (wer hat commitet), (b) Pull-Request-Review (wer hat approved), (c) Deploy-Log (wann auf welche Umgebung), (d) Secrets-Manager-Log (wer hat welches Secret abgerufen). Vier zusammen erzeugen den vollständigen Nachweis.

Aufgabe 8 – Anti-Muster im Agile Delivery

L2 • Anwendung

~ 18 min

YouTube-Suchquery: Scrum Kanban Anti Patterns Cargo Cult Friday Release Sprint Goal

Musterlösung

A – “Backlog-Abarbeitungs-Sprint”: *Anti-Muster:* kein Sprint Goal, kein Outcome. *Schaden:* Team verliert Fokus, Stakeholder bekommen Aufzählung statt Wert. *Sofort:* Sprint Goal als Outcome formulieren (“Befähige X”); *Strukturell:* Planning auf max. 2 h fokussieren, Backlog vorher refinieren.

B – “Status-Daily mit Detail-Moderation”: *Anti-Muster:* Scrum Master als Statusgeber. *Schaden:* Team-Selbstorganisation verloren; Daily wird zur Pflicht. *Sofort:* Scrum Master sitzt 5 Min still und beobachtet. *Strukturell:* Daily-Format auf 3 Fragen reduzieren (Was gestern? Was heute? Blocker?), Owner-Pattern (jede Person berichtet zur Sprint-Goal-Spalte).

C – “Board ohne WIP-Limit”: *Anti-Muster:* 47 Tickets parallel, Lead Time steigt. *Schaden:* nichts wird fertig, Multitasking. *Sofort:* WIP-Limit “In Progress” auf 5 setzen (Team-Größe × 1,5). *Strukturell:* Replenishment-Meeting; “erst leeren, dann ziehen”-Policy; Cycle Time pro Spalte messen.

D – “Freitag 17:00-Release ohne Rollback-Test”: *Anti-Muster:* “ship it”-Kultur ohne Sicherheitsnetz. *Schaden:* Incident im Wochenende; niemand reagiert, Kunden frustriert. *Sofort:* Friday-Release-Verbot (außer Hotfix). *Strukturell:* Rollback monatlich üben (“Game Day”); Canary Releases einführen; Deployment-Windows definieren (Mo – Do 08:00 – 15:00).

Niveau L3 – Management & Entscheidungsarchitektur

Aufgabe 9 – Fallstudie CityServices

L3 • Management

~ 45 min

YouTube-Suchquery: Public Sector Digital Approval Workflow Agile Scrum Kanban CI Release

Musterlösung

1. Hybrid-Ansatz: Scrum für Features (2-Wochen-Sprints) + Kanban-Lane für Incidents/Requests (WIP 2). Rollen: PO (Fachbereich), SM, Dev-Team, Plattform-Lead. Policies (siehe Aufgabe 4).

2. 3-Releases-Plan (12 Wochen):

- *MVP (Wo 1–4):* 3 Outcomes: digitaler Antrag + Statusanzeige + 1 Genehmigungsweg live.
- *Beta (Wo 5–8):* 3 Outcomes: Rollenmodell + SLA-Reminder + Audit Trail; 2. Genehmigungsweg.
- *Stabilisierung (Wo 9–12):* 3 Outcomes: Performance + Monitoring + Prozess-KPIs.

3. CI-Pipeline: Build □ Unit Tests □ Integration Tests □ Security Scan □ Deploy Staging □ Smoke Tests. 2 *Quality Gates*: Tests & Scan grün; Smoke + Health OK. *Release-Policy*: Mo–Do 08–15 Uhr, keine Friday-Releases außer Hotfix mit Approver.

4. IaC-Policy: Änderungen via Pull Request; 2-Augen-Freigabe; Umgebungen reproduzierbar; Secrets im Manager; Audit Log via Merge-Historie.

5. Fast vs. Safe:

- *Speed:* (a) MVP ohne vollständige Automatisierung aller Sonderfälle (manuelle Sonderpfade); (b) Integration Tests nur für Happy Path.
- *Safety:* (c) Security-Scan + Smoke-Test unverhandelbar; (d) Rollback-Plan und Feature-Toggle für Genehmigungswege.

6. Frühwarnsignale: (i) Change Failure Rate > 15 % in 2 aufeinanderfolgenden Wochen; (ii) Lead Time pro Item > 5 Tage. Beide auslösen Re-Planning.

Aufgabe 10 – Stop-the-Line

L3 • Management

~ 25 min

YouTube-Suchquery: Stop The Line Andon Continuous Delivery Emergency Change Management

Musterlösung**1. Drei Stop-the-Line-Regeln:**

1. *Tests rot* – keine Production-Releases, bis alle grün.
2. *Security-Finding offen* (kritisch oder hoch) – keine Production-Releases, bis adressiert oder dokumentiert akzeptiert.
3. *Production-Incident läuft* – keine Deployments während Incident-Bearbeitung.

2. Mandat “Stop”:

Das Mandat liegt beim *Platform Lead* bzw. *Release Manager*. Wenn das Mandat *unklar* verteilt ist: Eskalation läuft im Kreis (“Wer entscheidet?”), Releases werden trotzdem freigegeben, Incidents häufen sich. Wichtig: *kein einzelner Entwickler* sollte das Mandat haben – und *kein Manager* sollte ohne Plattform-Vertretung drüberentscheiden.

3. Emergency Change:

Ausnahme: *Hotfix bei Production-Outage*. Nachprozess: (a) Sofortige Notifikation an Platform Lead + SM, (b) Audit-Log mit “EMERGENCY”-Tag, (c) Post-Mortem innerhalb 5 Tagen, (d) ggf. Nachzieh-Tests in den nächsten Sprint.

4. Eskalation überstimmt Stop:

Wenn das Management eine Stop-Regel *politisch* überstimmt (“muss heute raus”), gilt:

- Schriftliche Risiko-Übernahme durch den Überstimmenden (E-Mail / Ticket).
- Audit-Log markiert das Release als “Override”.
- Folgen für Glaubwürdigkeit: wenn Überstimmungen *Routine* werden, ist die Stop-Regel tot – und das Team hat keine Sicherheit mehr. Faustregel: max. 1×/Halbjahr.

Aufgabe 11 – Transferprojekt – Agile + DevOps Operating Model

L3 • Management

~ 45 min

YouTube-Suchquery: Agile DevOps Operating Model DORA Team Topologies Quality Gates Roadmap

Musterlösung

1. Entscheidungsarchitektur:

- *Zentral*: Quality Gates, Security-Standards, IaC-Standards, Release-Policy, DORA-Definitionen.
- *Dezentral*: Team-Backlog, technische Umsetzung innerhalb der Standards, Sprint-Pläne.

2. Team-Topologie & Governance:

- *Stream-Aligned Teams* (PO, SM, Dev) liefern Features.
- *Platform Team* stellt CI/IaC/Secrets-Stack bereit ("X-as-a-Service" für Stream-Teams).
- *Enabling Team* (Security/Compliance) berät, nicht reviewt jeden Commit.
- RACI: PO accountable für Wert; Platform Lead accountable für Plattform-SLOs; Security accountable für Audit-Findings.
- Risikoakzeptanz: PO darf Feature-Toggles + Speed-Risiko entscheiden; Security darf Gate-Override blockieren.

3. Flow-Kennzahlen: DORA-4 (Lead Time, Deployment Frequency, MTTR, Change Failure Rate); Anti-Gaming siehe Aufgabe 6.

4. Quality Gate Strategy: Mindesttests (Unit + Integration auf Happy Path), Security-Scan, Release-Policy (Deployment-Windows), Audit Trail; Emergency Change geregelt mit Post-Mortem-Pflicht.

5. Roadmap (16 Wochen):

- Wo 1 – 4: Pilotteam (1 Stream-Team), Plattform-MVP, Charter publiziert.
- Wo 5 – 8: Plattform skaliert auf 3 Teams, DORA-Messung produktiv.
- Wo 9 – 12: Skalierung auf 6 Teams, Audit-Vorbereitung.
- Wo 13 – 16: Sustain, Optimierung, Lessons Learned.
- Enablement: L1 (Lesen + Daily), L2 (Pipeline + IaC), L3 (Governance + Architecture).

6. Entscheidungen unter Unsicherheit:

1. *Release-Frequenz-Ziel*: 1×/Woche pro Team in den ersten 8 Wochen, danach Steigerung auf 2×/Woche. *Risiko*: Überlastung Plattform-Team. *Frühwarnsignal*: Plattform-Incident-Rate > 2/Woche.
2. *Standardisierung vs. Autonomie*: Standards für Pipeline-Stufen, Security, IaC; Autonomie für Tech-Stack innerhalb der Standards. *Akzeptanzstrategie*: Standards zusammen mit Tech-Leads formulieren. *Trigger zum Nachschärfen*: wenn 2+ Teams das gleiche Workaround entwickeln.

Abschluss

Die Musterlösungen sind eine Diskussionsbasis. Heute besonders: Habe ich *Agile* als Steuerung verstanden – oder als Ritual?

Nächster Schritt

Bringen Sie ins Coaching: Welche *eine* Stop-the-Line-Regel würden Sie morgen in Ihrer Organisation einführen? Wer hätte das Mandat? Was würde im ersten Override passieren? Sind Ihre DORA-Kennzahlen *gekoppelt* – oder optimieren sie sich gegenseitig zu Tode?